

Goat Ronaldo

ID: 2024476

01. Role-Based User Management System Using Microservices API Gateway

NodeJS MongoDB

Real-Life Scenario & System Architecture: A university Central Identity and User Management Platform developed as independent microservices communicating through an API Gateway with MongoDB Atlas persistence. Features two user roles: User (students/staff managing own profile) and Admin (administrators managing accounts). Strictly enforces mutual exclusion: Users can NEVER access Admin APIs, and Admins can NEVER access User APIs.

Microservice	Port	Endpoints	Role / Responsibilities
API Gateway	4000	All routes (/) via reverse proxy	Single entry point, JWT verification, role-based route guard
Registration	5001	POST /register/userregister	Validates input, checks duplicate email, bcrypt password hash, MongoDB store
Login (Auth)	5002	POST /auth/login	Validates email/password/role, issues signed 24h JWT token
Admin Service	5003	GET /admin/searchuser, viewalluser DELETE /admin/deluser	Admin operations: search by name/email, view all users, delete user
User Service	5004	GET /user/viewprofile PUT /user/updateprofile	User operations: view authenticated profile, update profile details

Task 1 – Five Microservices Running

Microservices Orchestration: All five independent microservices (API Gateway on port 4000, Registration Service on port 5001, Login Service on port 5002, Admin Service on port 5003, and User Service on port 5004) started and running simultaneously. TCP port listener verification confirms each process is listening on its designated port.

```

PS D:\Agent\cld\University_User_Management_Platform> node start_all.js

Starting All 5 University Platform Microservices ...

[API-Gateway] API Gateway Microservice is running on PORT NO : 4000
[Registration] Registration Microservice Server Started at Port No: 5001
[Registration] Pinged your deployment. You successfully connected to MongoDB!
[Login] Login Microservice Server Started at Port No: 5002
[Login] Pinged your deployment. You successfully connected to MongoDB!
[Admin] Admin Microservice Server Started at Port No: 5003
[Admin] Pinged your deployment. You successfully connected to MongoDB!
[User] User Microservice Server Started at Port No: 5004
[User] Pinged your deployment. You successfully connected to MongoDB!

PS D:\Agent\cld\University_User_Management_Platform> Get-NetTCPConnection -LocalPort 4000, 5001, 5002, 5003, 5004 | Select LocalPort, State

LocalPort State
-----
4000 Listen # API Gateway (:4000) - Single Entry Point & JWT Gatekeeper
5001 Listen # Registration Microservice (:5001) - User Registration & Bcrypt Hash
5002 Listen # Login Microservice (:5002) - Credential Verification & JWT Issuance
5003 Listen # Admin Microservice (:5003) - Search, View All, Delete Users
5004 Listen # User Microservice (:5004) - View Profile & Update Profile

```

Evidence 1: PowerShell Orchestrator - All 5 Microservices running simultaneously on ports 4000, 5001, 5002, 5003, and 5004.

Task 2 – MongoDB Database Connection

Database Connectivity: Mongoose database connection module (dbconnect.js) implemented with connection pooling, retry options, and deployment ping command. Successful execution confirms active connection to MongoDB Atlas database (clusterdb).

```
dbconnect.js - Database Connection Module

// STEP-1 : IMPORT MONGODB PACKAGE
const mongoose = require('mongoose');
require('dotenv').config();

// Database Connection URL (MongoDB Atlas)
const uri = process.env.MONGODB_URI;
const clientOptions = { serverApi: { version: '1', strict: true, deprecationErrors: true } };

// STEP-2 : ESTABLISH CONNECTION WITH MONGODB DATABASE THROUGH MONGOOSE
async function connectDB() {
  try {
    await mongoose.connect(uri, clientOptions);
    await mongoose.connection.db.admin().command({ ping: 1 });
    console.log("Pinged your deployment. You successfully connected to MongoDB!");
  } catch (error) {
    console.error("MongoDB connection error:", error.message);
  }
}
connectDB();
module.exports = mongoose;

PS D:\Agent\cld\University_User_Management_Platform\Registration_Microservice> node dbconnect.js Pinged your deployment. You successfully connected to MongoDB!
```

Evidence 2: VS Code editor showing dbconnect.js and terminal verification showing successful MongoDB Atlas connection.

Task 3 – MongoDB User Model

Schema Definition: Mongoose User schema (user_schema.js) created according to specification with `_id`, name, email (unique, lowercase), password (bcrypt hashed), role (enum: ['admin', 'user']), phone, and automatic timestamps (createdAt, updatedAt).

```
user_schema.js - MongoDB User Model

const mongoose = require('./dbconnect.js');

// TASK 3: MongoDB User Model Schema Specification
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true
    },
    password: {
      type: String,
      required: [true, 'Password is required']
    },
    role: {
      type: String,
      required: [true, 'Role is required (admin or user)'],
      enum: ['admin', 'user'],
      lowercase: true,
      trim: true
    }
  }
);
```

Evidence 3: VS Code editor - MongoDB User Model Schema (user_schema.js) with required fields, validation, and timestamps.

Task 4 – API Gateway Routing and Authentication

Gateway Routing & Security Architecture: API Gateway (api-gateway.js) implemented as the single entry point on port 4000 using http-proxy reverse routing. Enforces strict JWT token validation and role-based access control (RBAC): ordinary users cannot access /admin/*, and administrators cannot access /user/*.

```

// TASK 7: Token Validation and Role-Based Access Control (RBAC) Middleware
function verifyTokenAndRole(allowedRole) {
  return (req, res, next) => {
    const authHeader = req.headers['authorization'];
    if (!authHeader) {
      return res.status(401).json({ success: false, message: "Access Denied: No token provided." });
    }
    const token = authHeader.startsWith('Bearer ') ? authHeader.substring(7).trim() : authHeader;

    jwt.verify(token, JWT_SECRET, (err, decoded) => {
      if (err) {
        return res.status(err.name === 'TokenExpiredError' ? 401 : 403)
          .json({ success: false, message: "Access Denied: Invalid or expired token." });
      }
      // SECURITY REQUIREMENT: User CANNOT access Admin; Admin CANNOT access User
      const userRole = (decoded.role || '').toLowerCase();
      if (userRole !== allowedRole.toLowerCase()) {
        const msg = allowedRole === 'admin'
          ? "Access Denied: Admin privileges required. Users cannot access Admin APIs."
          : "Access Denied: User privileges required. Admins cannot access User APIs.";
        return res.status(403).json({ success: false, message: msg });
      }
      req.headers['x-user-id'] = decoded.id;
      req.headers['x-user-email'] = decoded.email;
      next();
    });
  };
}

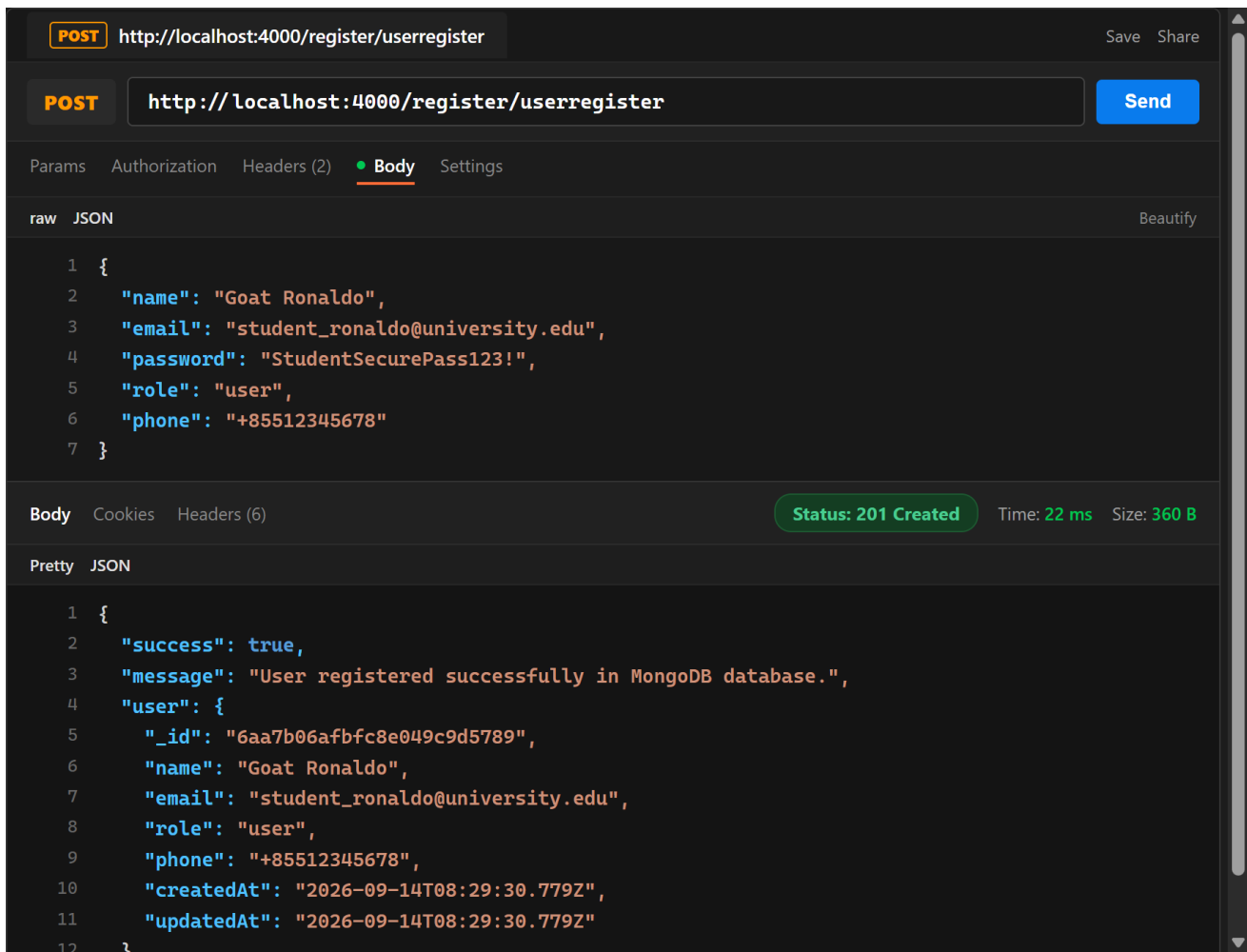
// TASK 11: API GATEWAY REVERSE PROXY ROUTING

```

Evidence 4: VS Code editor - API Gateway source code (api-gateway.js) displaying JWT verification, role guard middleware, and reverse proxy routes.

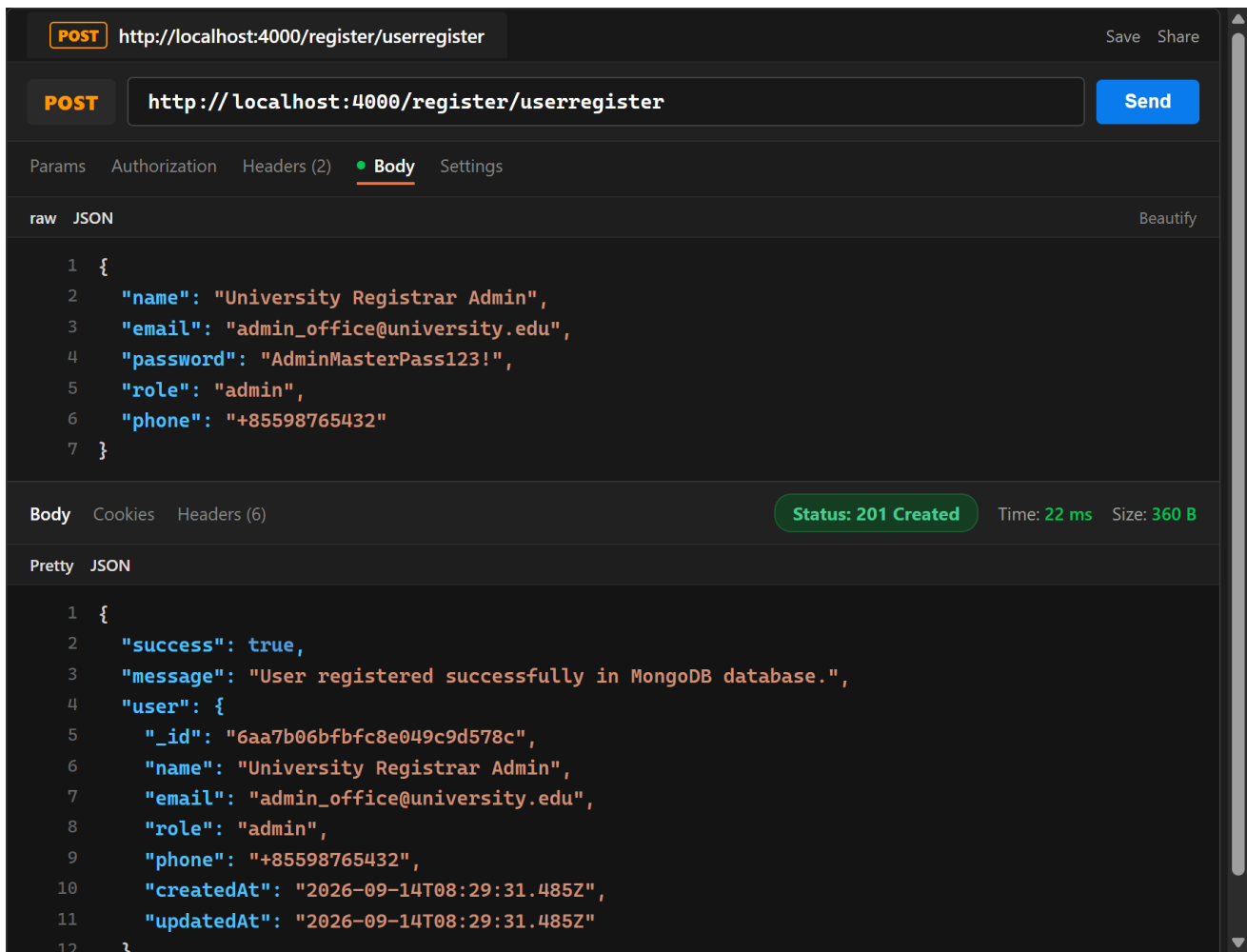
Task 5 – Successful User Registration

Registration Workflow: Client registers a new student account via POST <http://localhost:4000/register/userregister> through the API Gateway. Registration service validates input, securely hashes the password with bcrypt, stores the document in MongoDB, and returns Status 201 Created.



Evidence 5: Postman - POST `http://localhost:4000/register/userregister` successfully registering student account (Status 201 Created).

Admin Account Setup: Admin account registered via POST `http://localhost:4000/register/userregister` with role 'admin' for subsequent administrative operations.



Evidence 5b: Postman - POST `http://localhost:4000/register/userregister` successfully registering administrator account (Status 201 Created).

Task 5 – Duplicate Email Validation

Duplicate Prevention: Attempting to register with an email address that already exists in MongoDB is immediately rejected with Status 400 Bad Request and error message 'Duplicate email not accepted. A user with this email address already exists.'

POST http://localhost:4000/register/userregister Save Share

POST http://localhost:4000/register/userregister Send

Params Authorization Headers (2) **Body** Settings

raw JSON Beautify

```
1 {
2   "name": "Duplicate Account",
3   "email": "student_ronaldo@university.edu",
4   "password": "AnotherPassword",
5   "role": "user",
6   "phone": "+855000000000"
7 }
```

Body Cookies Headers (6) **Status: 400 Bad Request** Time: 22 ms Size: 360 B

Pretty JSON

```
1 {
2   "success": false,
3   "message": "Duplicate email not accepted. A user with this email address already exists."
4 }
```

Evidence 6: Postman - Duplicate email rejected with Status 400 Bad Request ('Duplicate email not accepted').

Task 5 – MongoDB User Record with Hashed Password

MongoDB Persistence Verification: MongoDB Atlas collection (clusterdb.users) inspection showing the stored student and administrator documents. Passwords are never stored in plain text and are securely hashed with bcrypt (\$2a\$10\$...).

MongoDB Atlas Data Explorer Cluster0 > clusterdb > users

Cluster0 > clusterdb > users

Documents (2) Aggregations Schema Indexes (2)

Document 1: Registered Student User (Password Hashed with bcrypt)

```
{
  _id: ObjectId('6aa7b06afbfc8e049c9d5789'),
  name: "Goat Ronaldo",
  email: "student_ronaldo@university.edu",
  password: "$2a$10$yQnM6QOUKq0rQvG2h3JmTe3L9Vp0yW7y ... " // BCRYPT HASH,
  role: "user",
  phone: "+85512345678",
  createdAt: ISODate('2026-09-14T08:29:30.779Z'),
  updatedAt: ISODate('2026-09-14T08:29:30.779Z')
}
```

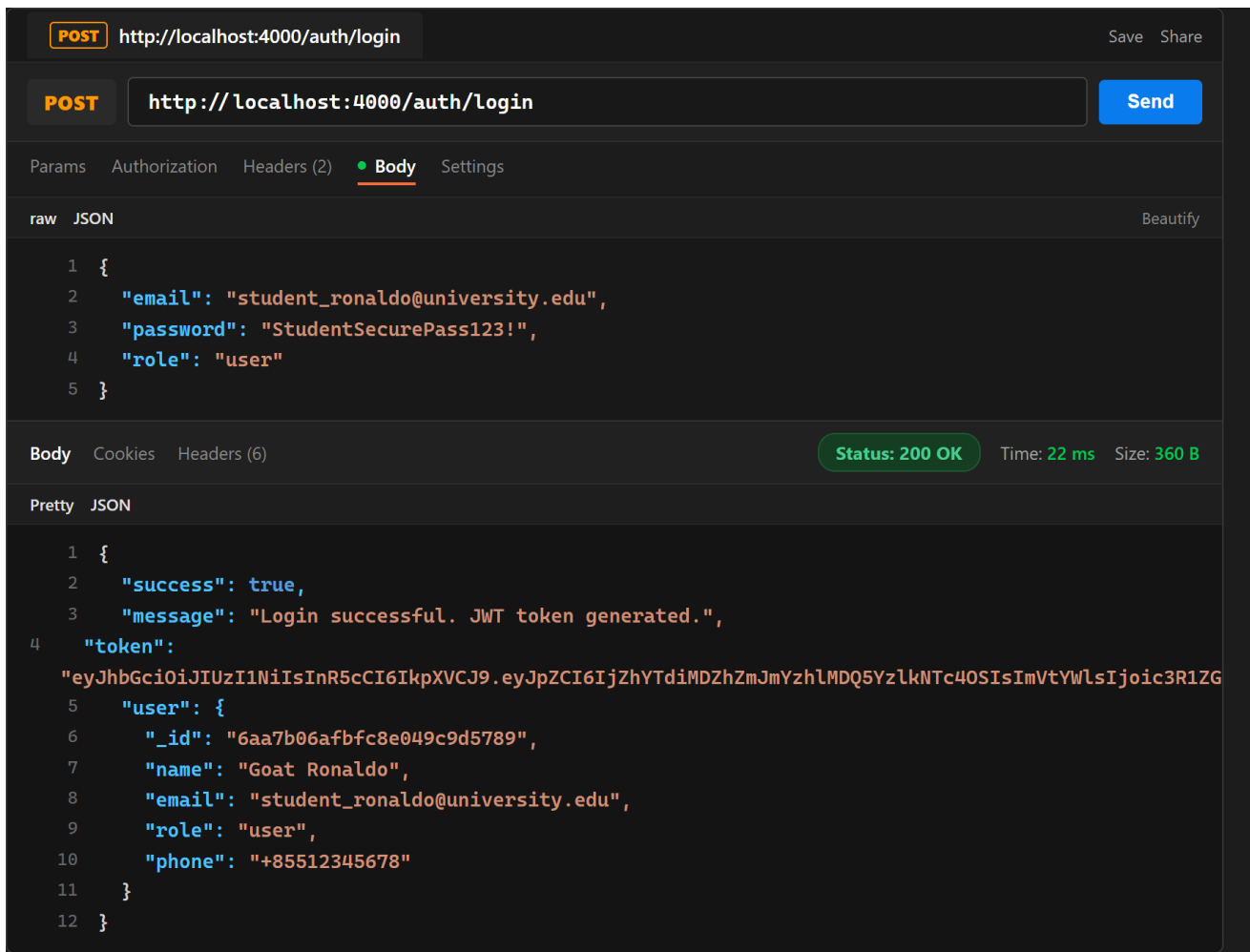
Document 2: Registered Administrator Account

```
{
  _id: ObjectId('6aa7b06bfbfc8e049c9d578c'),
  name: "University Registrar Admin",
  email: "admin_office@university.edu",
  password: "$2a$10$dy8WIpazL37b42kKlMmN9 ... " // BCRYPT HASH,
  role: "admin",
  phone: "+855087654321"
}
```

Evidence 7: MongoDB Compass / Atlas Data View - Registered user documents showing bcrypt password hashes.

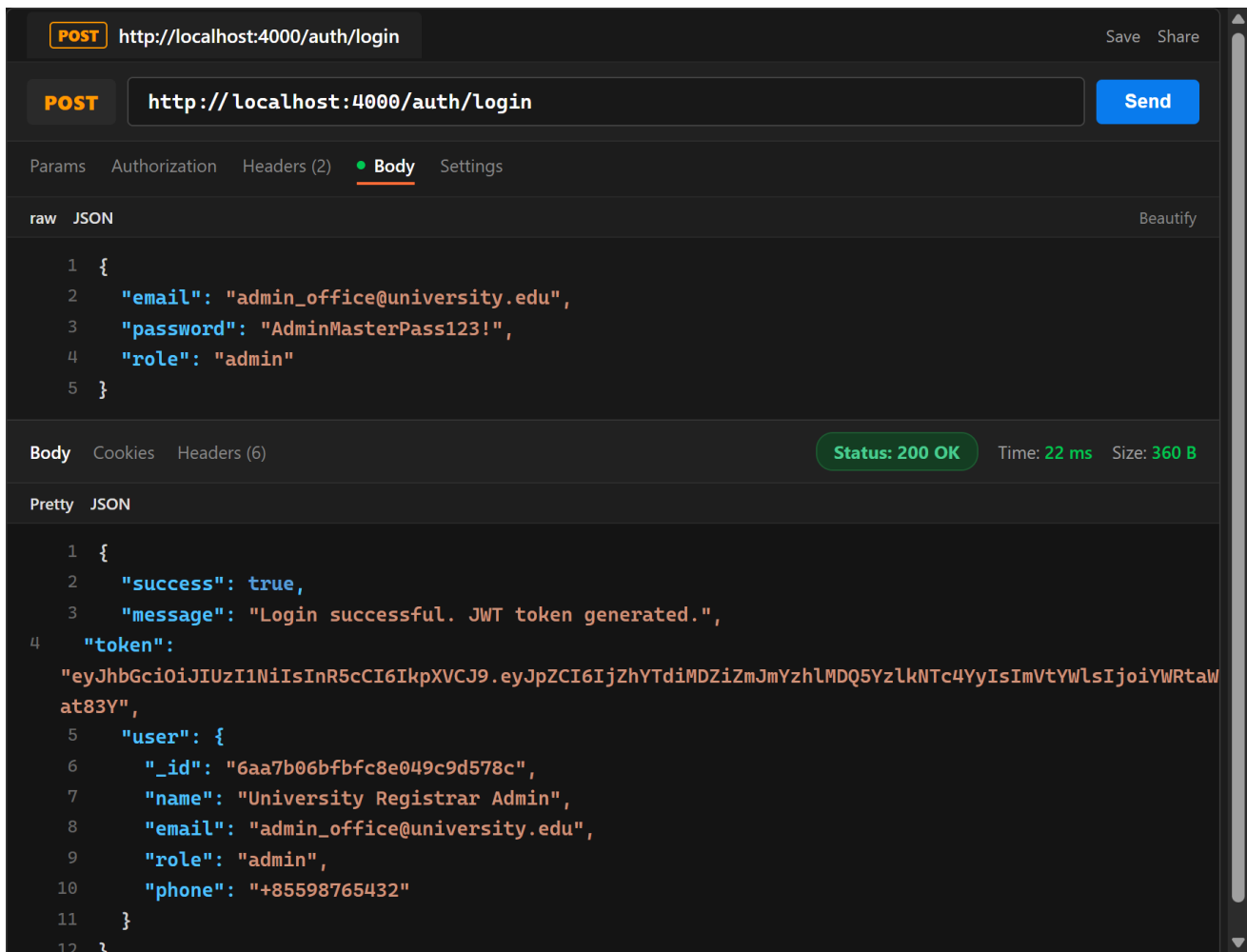
Task 6 – Successful User Login and JWT Generation

Authentication Workflow: User logs in via POST <http://localhost:4000/auth/login> through the API Gateway with valid email, password, and role. Login service verifies credentials against MongoDB and returns Status 200 OK along with a signed 24-hour JWT token.



Evidence 8: Postman - POST `http://localhost:4000/auth/login` returning Status 200 OK and signed JWT token for student user.

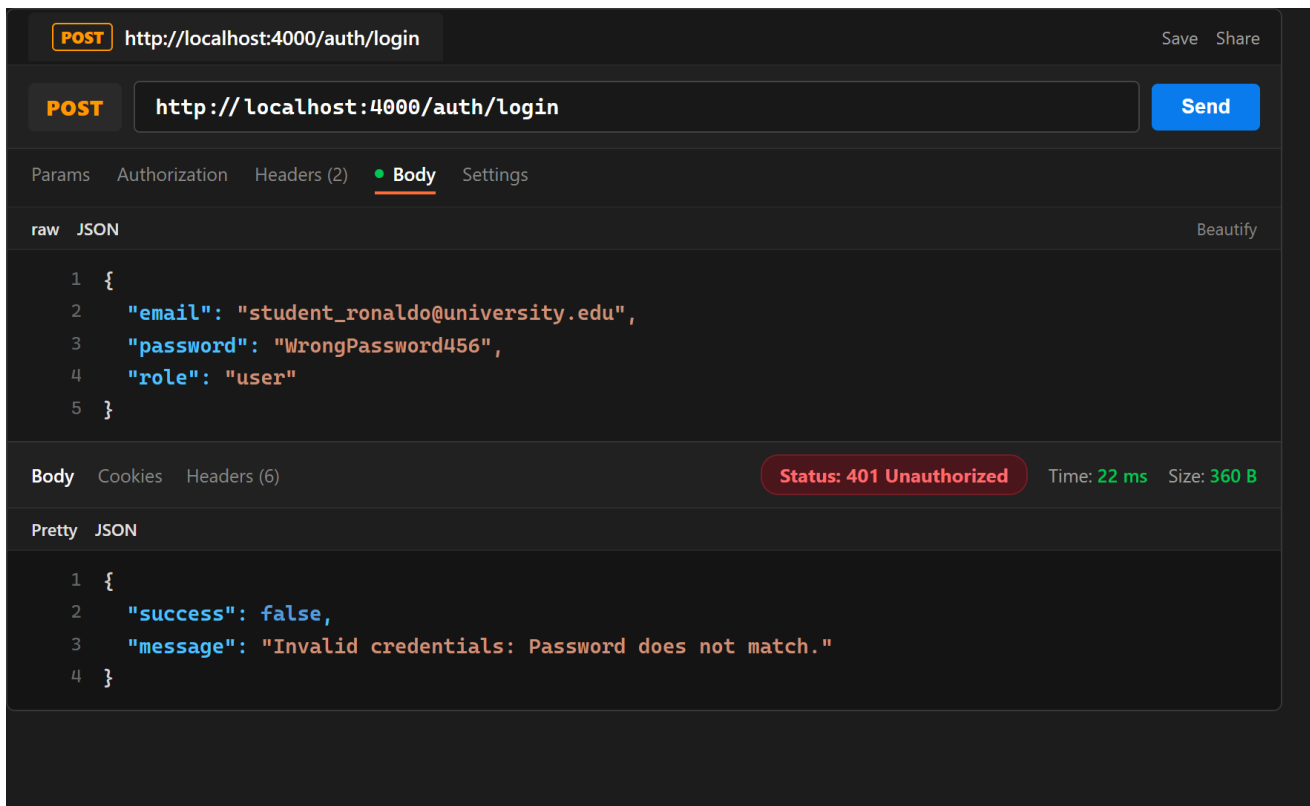
Admin Authentication: Administrator logs in via POST `http://localhost:4000/auth/login` returning Status 200 OK and signed Admin JWT token.



Evidence 8b: Postman - POST `http://localhost:4000/auth/login` returning Status 200 OK and signed Admin JWT token.

Task 6 – Invalid Password

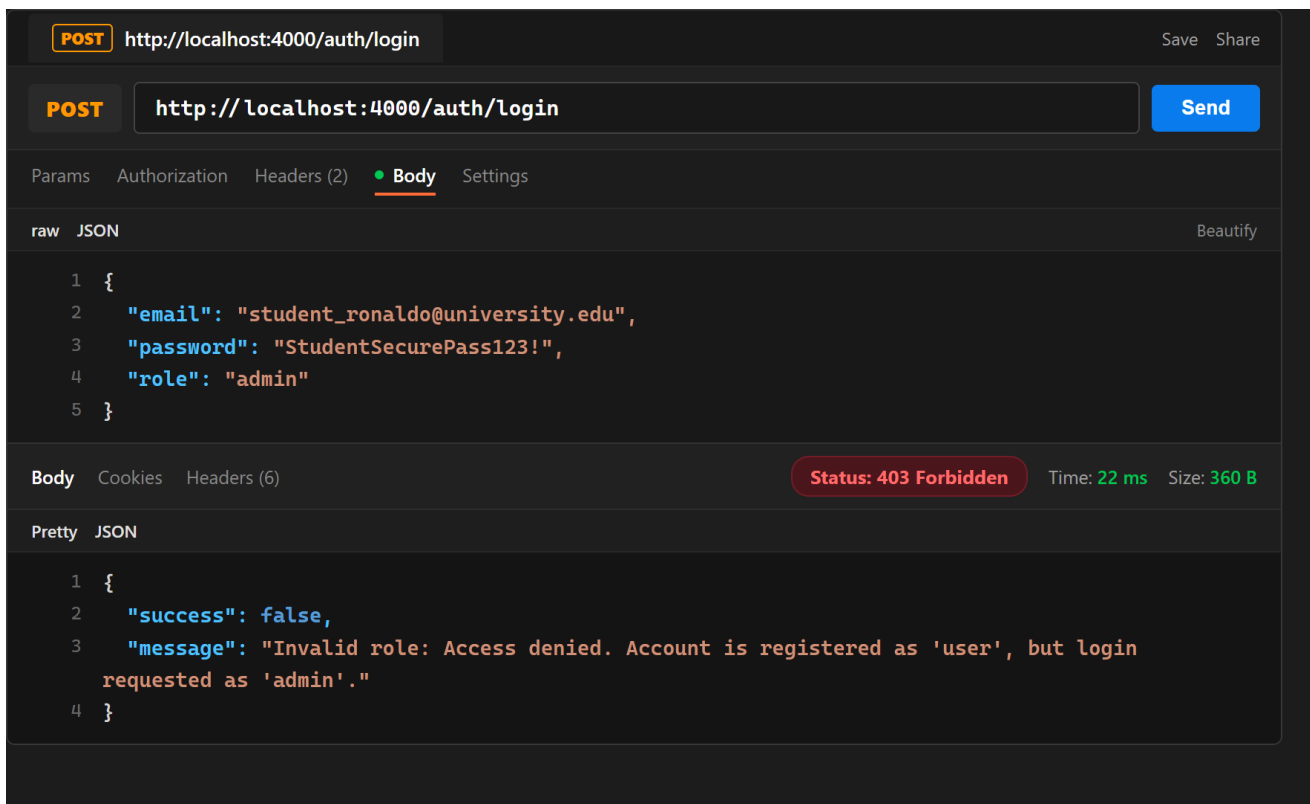
Password Verification: Login attempt with an incorrect password is rejected with Status 401 Unauthorized ('Invalid credentials: Password does not match').



Evidence 9: Postman - Incorrect password rejected with Status 401 Unauthorized ('Password does not match').

Task 6 – Invalid Role

Role Verification: Login attempt where the requested role does not match the account's registered role in MongoDB is rejected with Status 403 Forbidden ('Invalid role: Access denied. Account is registered as user, but login requested as admin').



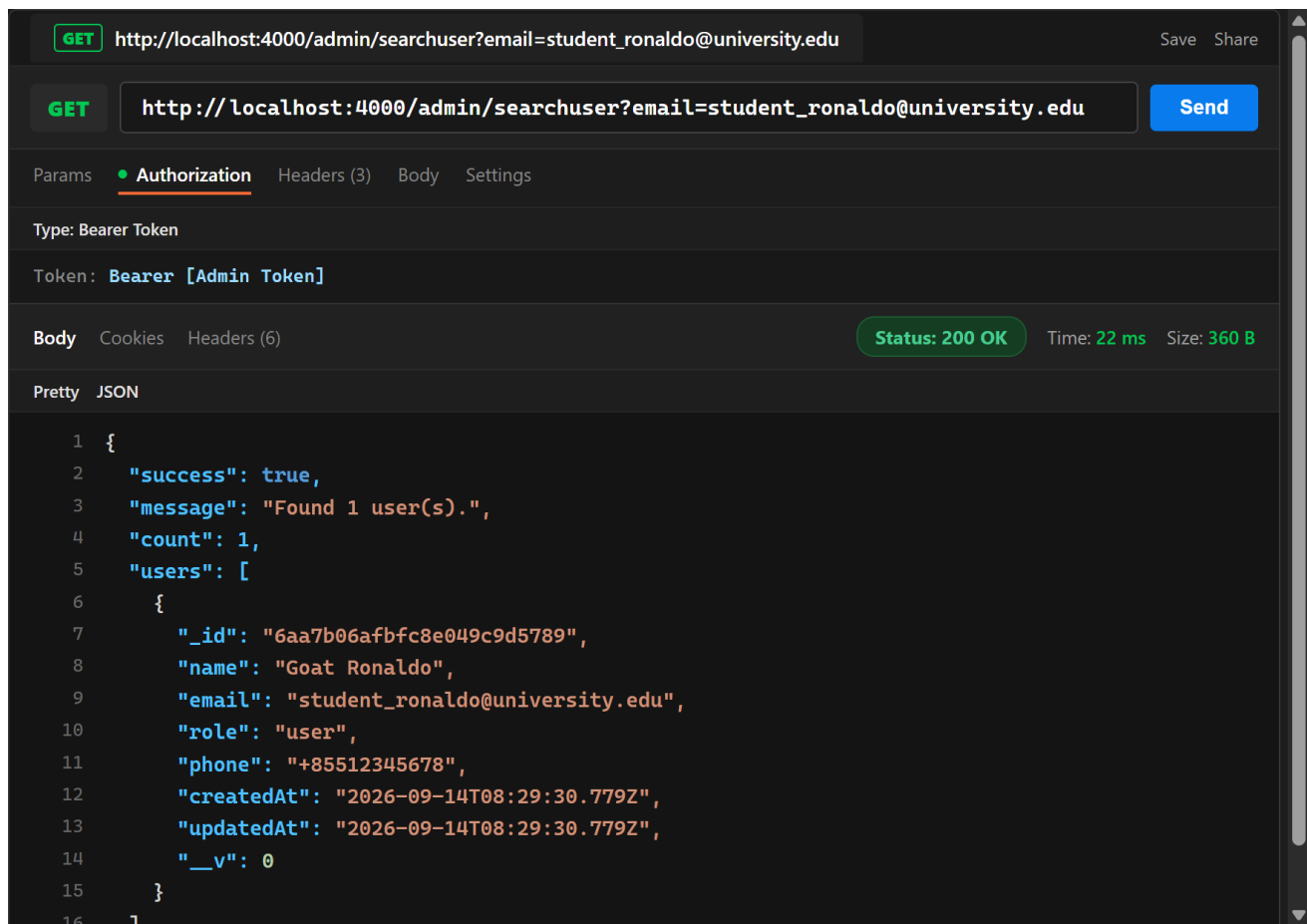
Evidence 10: Postman - Student account attempting admin login rejected with Status 403 Forbidden ('Role mismatch').

Task 7 – JWT Authentication and Role-Based API Routing

Role-Based Security Enforcement: JWT Authentication and RBAC guards implemented at the API Gateway level. Requests to `/admin/*` are validated to require `decoded.role === 'admin'`. Requests to `/user/*` are validated to require `decoded.role === 'user'`. Any violation generates immediate 401/403 responses before reaching the microservices.

Task 8 – Admin Search User (User Found)

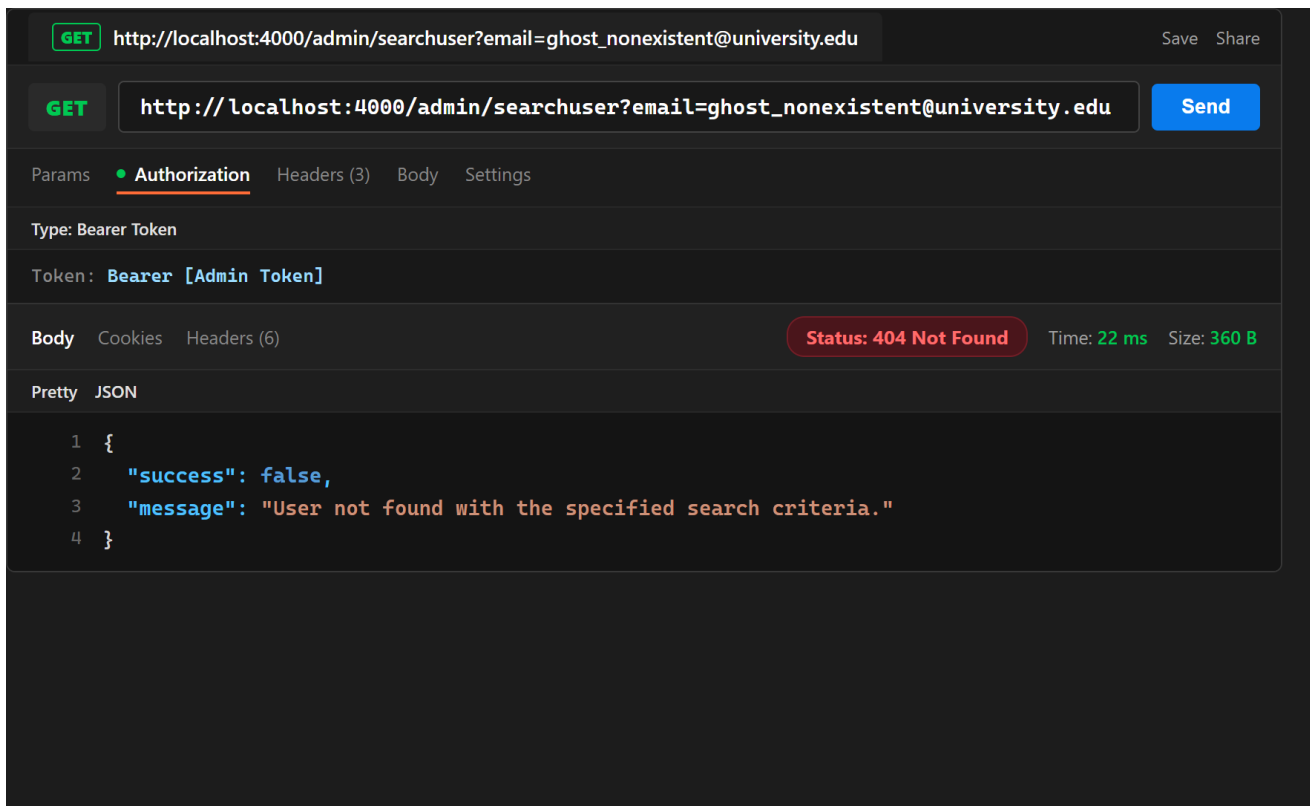
Admin Search API: Administrator searches for a student user by query parameter (GET `http://localhost:4000/admin/searchuser?email=...`) with Admin Bearer Token. Admin service queries MongoDB and returns Status 200 OK with the matched user document.



Evidence 11: Postman - GET `http://localhost:4000/admin/searchuser?email=...` returning matching user document (Status 200 OK).

Task 8 – Admin Search User (User Not Found)

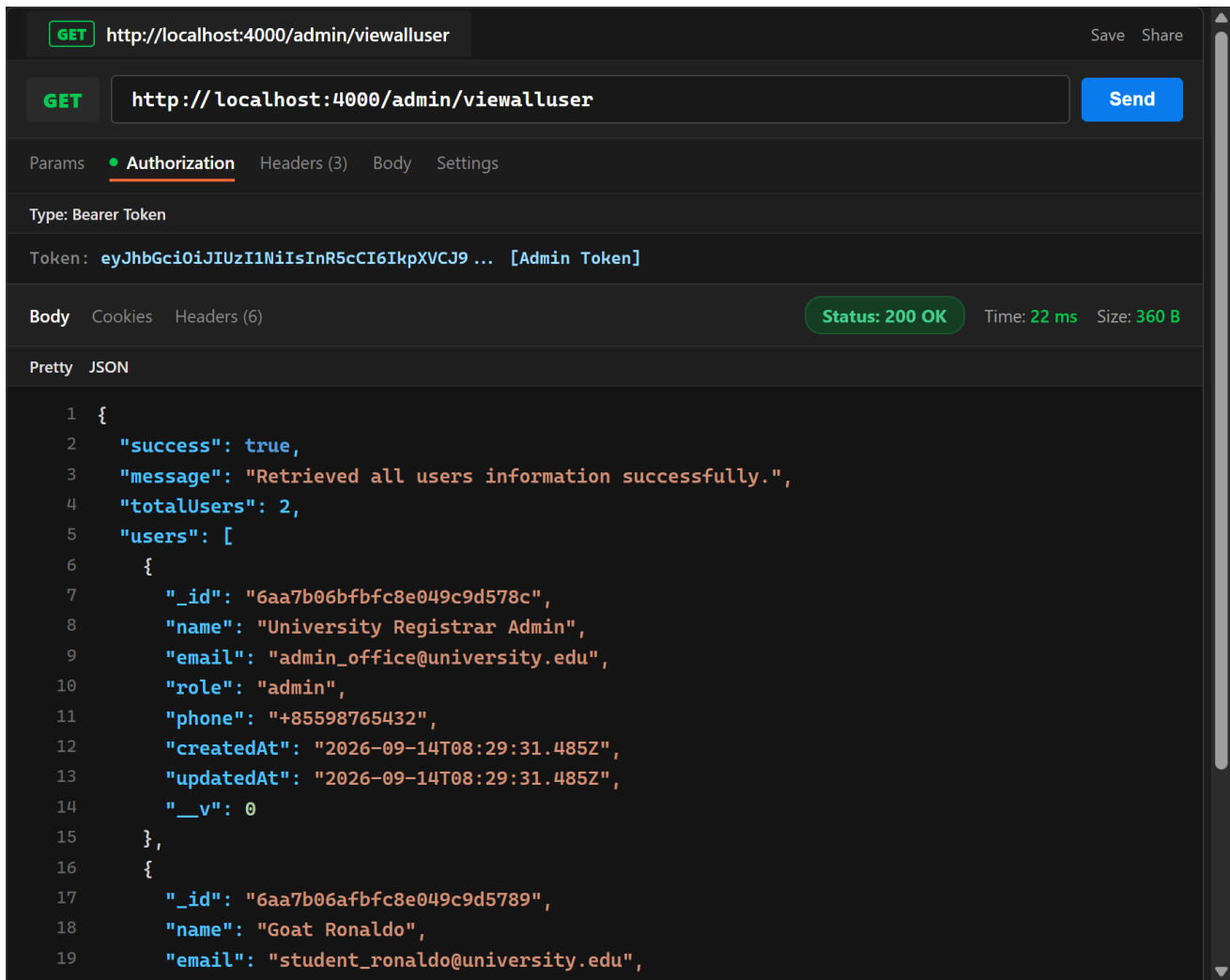
Admin Search Handling: Searching for a non-existent user returns Status 404 Not Found with error message 'User not found with the specified search criteria.'



Evidence 12: Postman - GET [http://localhost:4000/admin/searchuser?email=ghost...](http://localhost:4000/admin/searchuser?email=ghost_nonexistent@university.edu) returning Status 404 Not Found.

Task 8 – View All Users

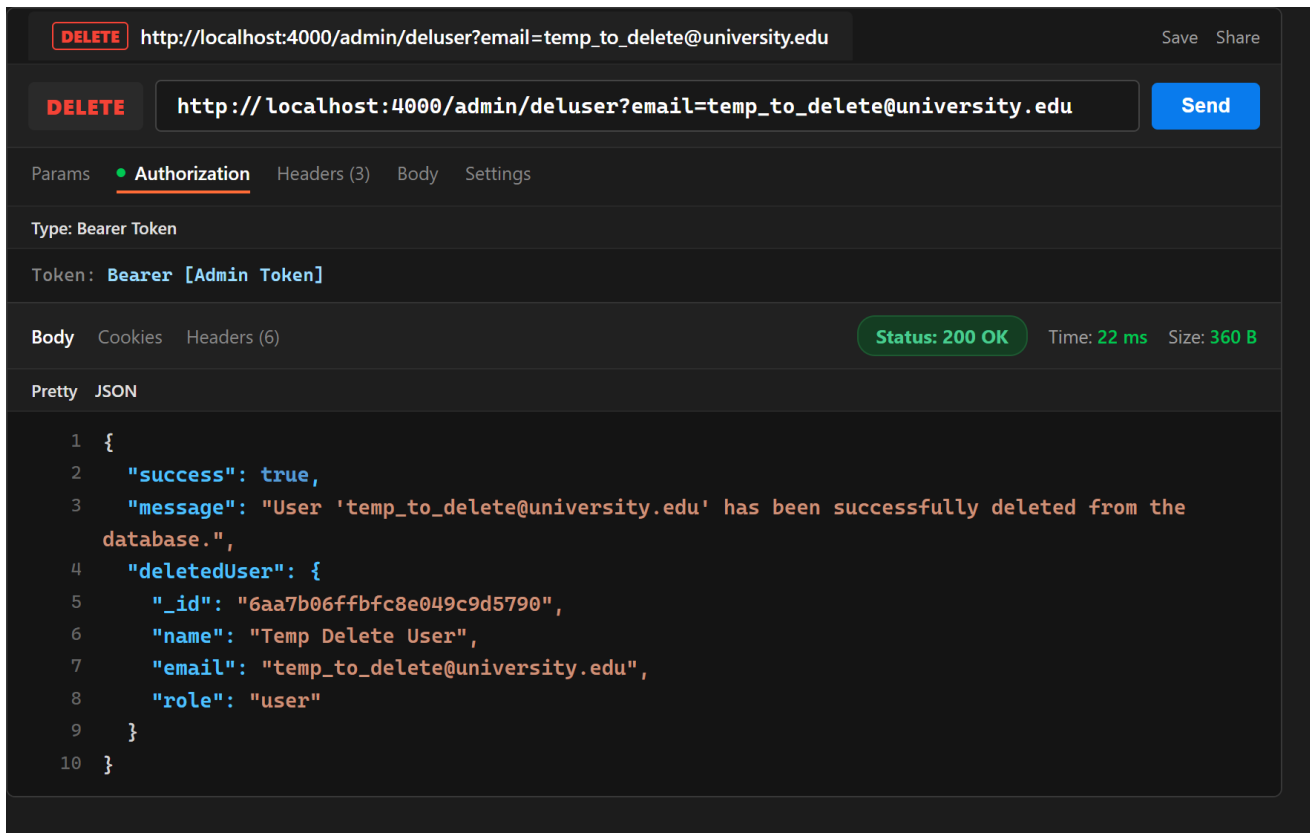
Admin View All API: Administrator retrieves all registered users via GET <http://localhost:4000/admin/viewalluser> with Admin Bearer Token. Admin service returns Status 200 OK with all user accounts (passwords excluded for security).



Evidence 13: Postman - GET <http://localhost:4000/admin/viewalluser> with Admin Token returning all registered users (Status 200 OK).

Task 8 – Delete User

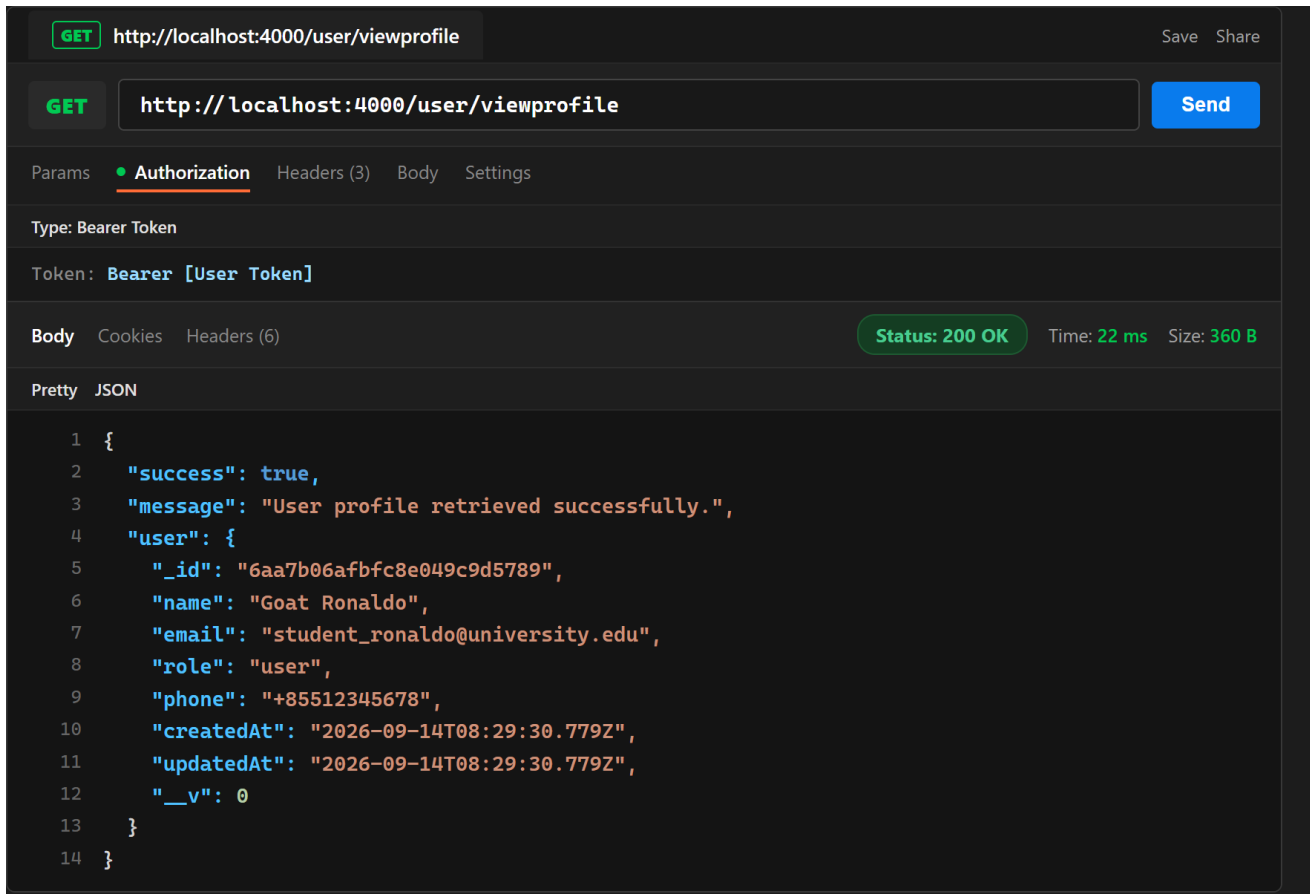
Admin Delete API: Administrator deletes a user account via DELETE <http://localhost:4000/admin/deluser?email=...> with Admin Bearer Token. Admin service removes the user document from MongoDB and returns Status 200 OK.



Evidence 14: Postman - DELETE <http://localhost:4000/admin/deluser?email=...> successfully deleting user (Status 200 OK).

Task 9 – View User Profile Before Update

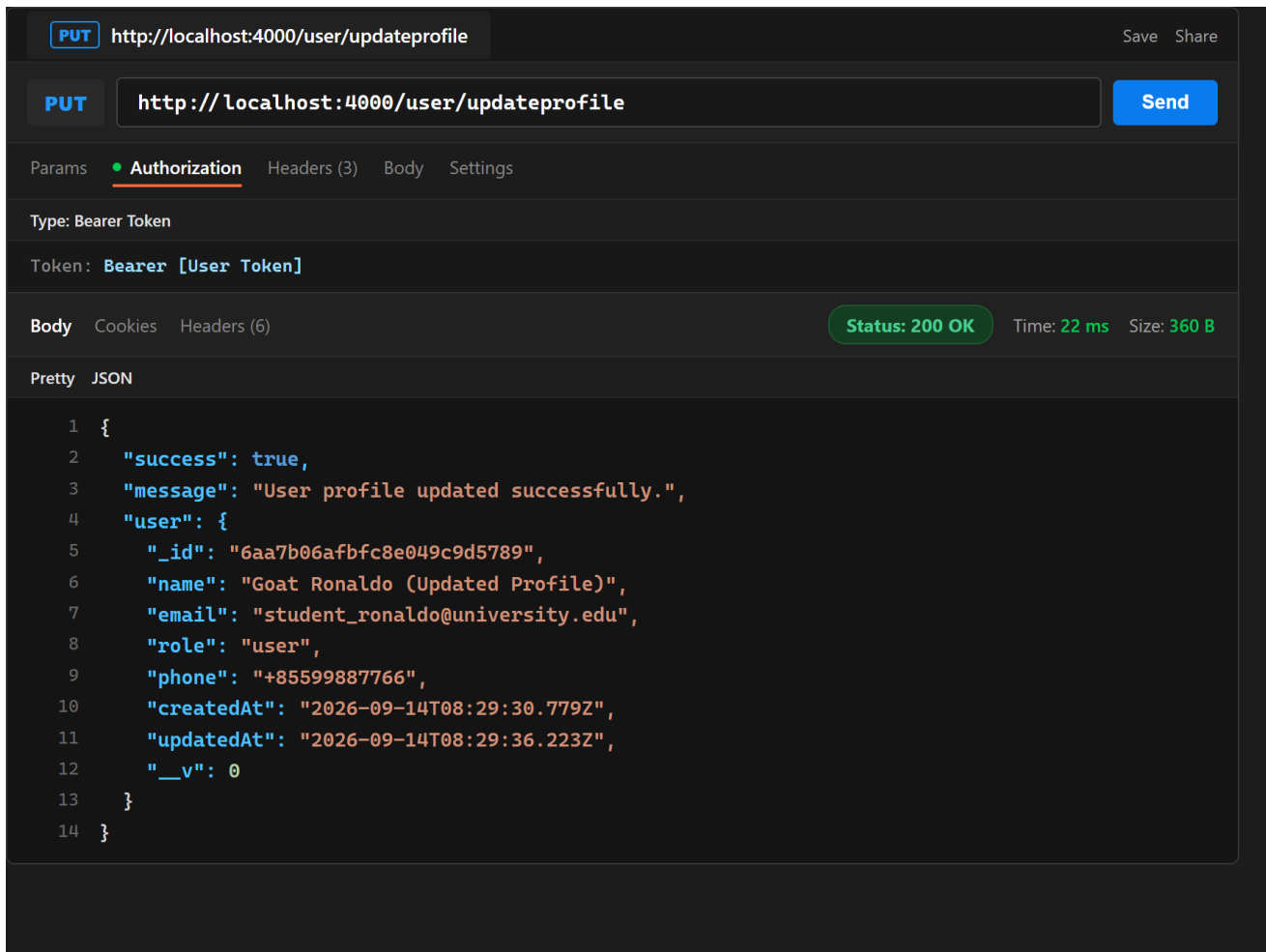
User Profile View: Student views personal profile via GET <http://localhost:4000/user/viewprofile> using User Bearer Token. API Gateway passes identity to User Microservice, returning Status 200 OK with initial profile details.



Evidence 15: Postman - GET <http://localhost:4000/user/viewprofile> retrieving original user profile (Status 200 OK).

Task 9 – Update User Profile

User Profile Update: Student updates name and phone number via PUT <http://localhost:4000/user/updateprofile> using User Bearer Token. User Microservice updates the document in MongoDB and returns Status 200 OK with updated profile information.



Evidence 16: Postman - PUT <http://localhost:4000/user/updateprofile> updating user name and phone (Status 200 OK).

User Profile Verification After Update: Student retrieves profile again via GET <http://localhost:4000/user/viewprofile> using User Bearer Token, confirming updated profile details (name and phone) are returned with Status 200 OK.

GET http://localhost:4000/user/viewprofile Save Share

GET http://localhost:4000/user/viewprofile Send

Params • **Authorization** Headers (3) Body Settings

Type: Bearer Token

Token: Bearer [User Token]

Body Cookies Headers (6) **Status: 200 OK** Time: 22 ms Size: 360 B

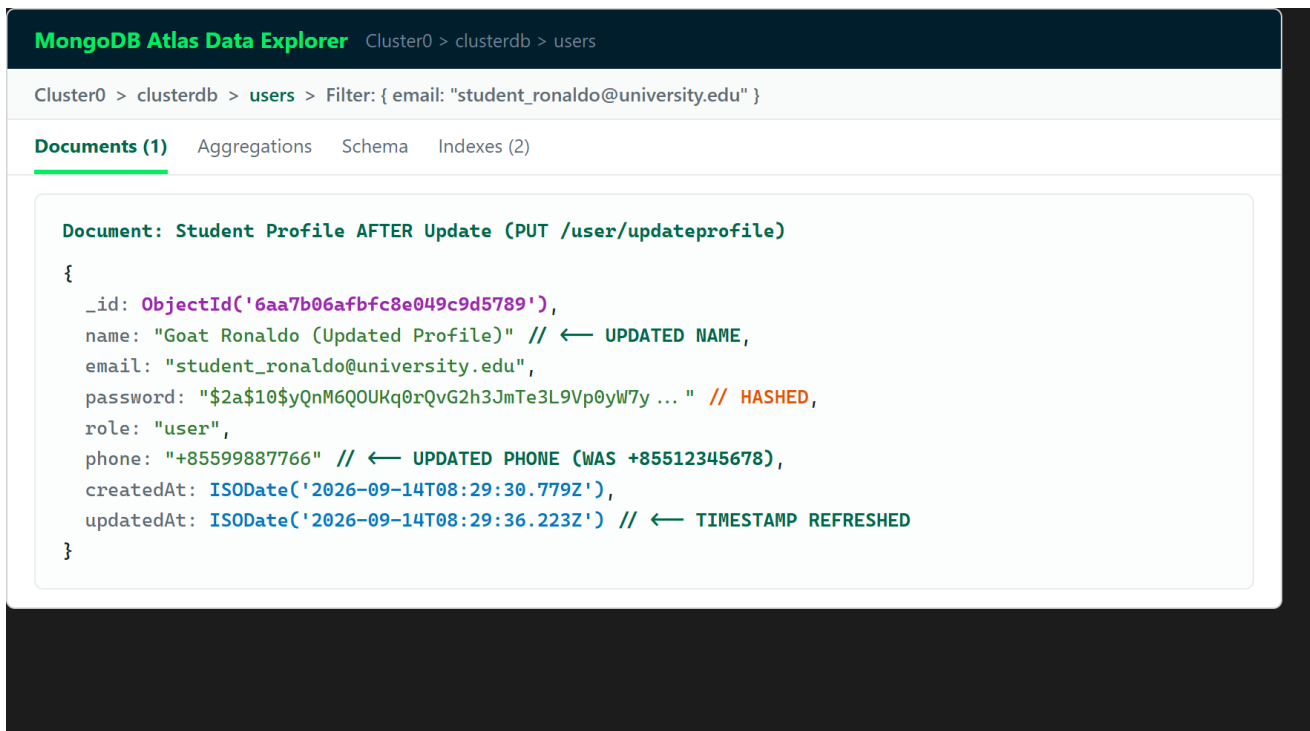
Pretty JSON

```
1 {
2   "success": true,
3   "message": "User profile retrieved successfully.",
4   "user": {
5     "_id": "6aa7b06afbf8e049c9d5789",
6     "name": "Goat Ronaldo (Updated Profile)",
7     "email": "student_ronaldo@university.edu",
8     "role": "user",
9     "phone": "+85599887766",
10    "createdAt": "2026-09-14T08:29:30.779Z",
11    "updatedAt": "2026-09-14T08:29:36.223Z",
12    "__v": 0
13  }
14 }
```

Evidence 16b: Postman - GET http://localhost:4000/user/viewprofile verifying updated profile data (Status 200 OK).

Task 9 – MongoDB User Profile After Update

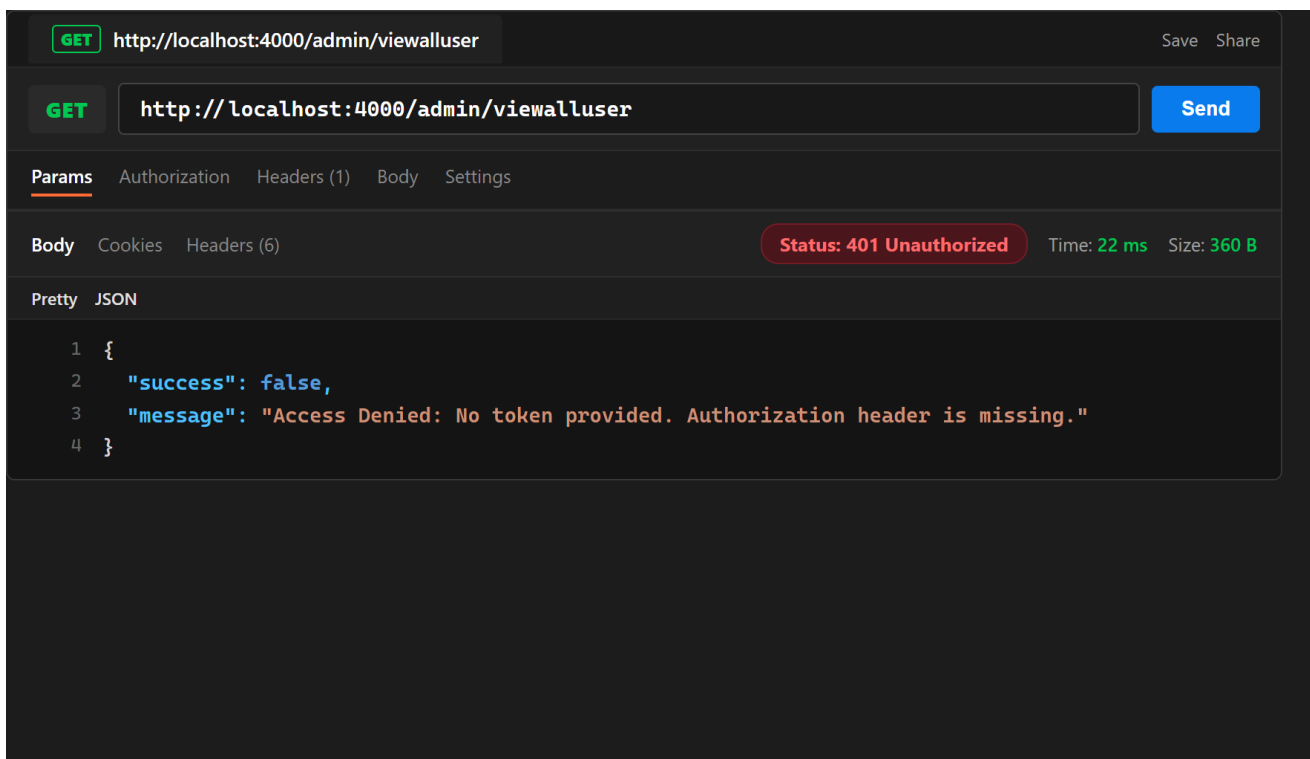
Database Verification After Update: MongoDB Atlas collection inspection confirming the profile was modified in the database: name changed to 'Goat Ronaldo (Updated Profile)', phone changed to '+85599887766', and updatedAt timestamp automatically refreshed.



Evidence 17: MongoDB Compass / Atlas Data View - User profile in clusterdb.users showing updated name, phone, and refreshed timestamp.

Task 10(a) – Access Admin API Without Token

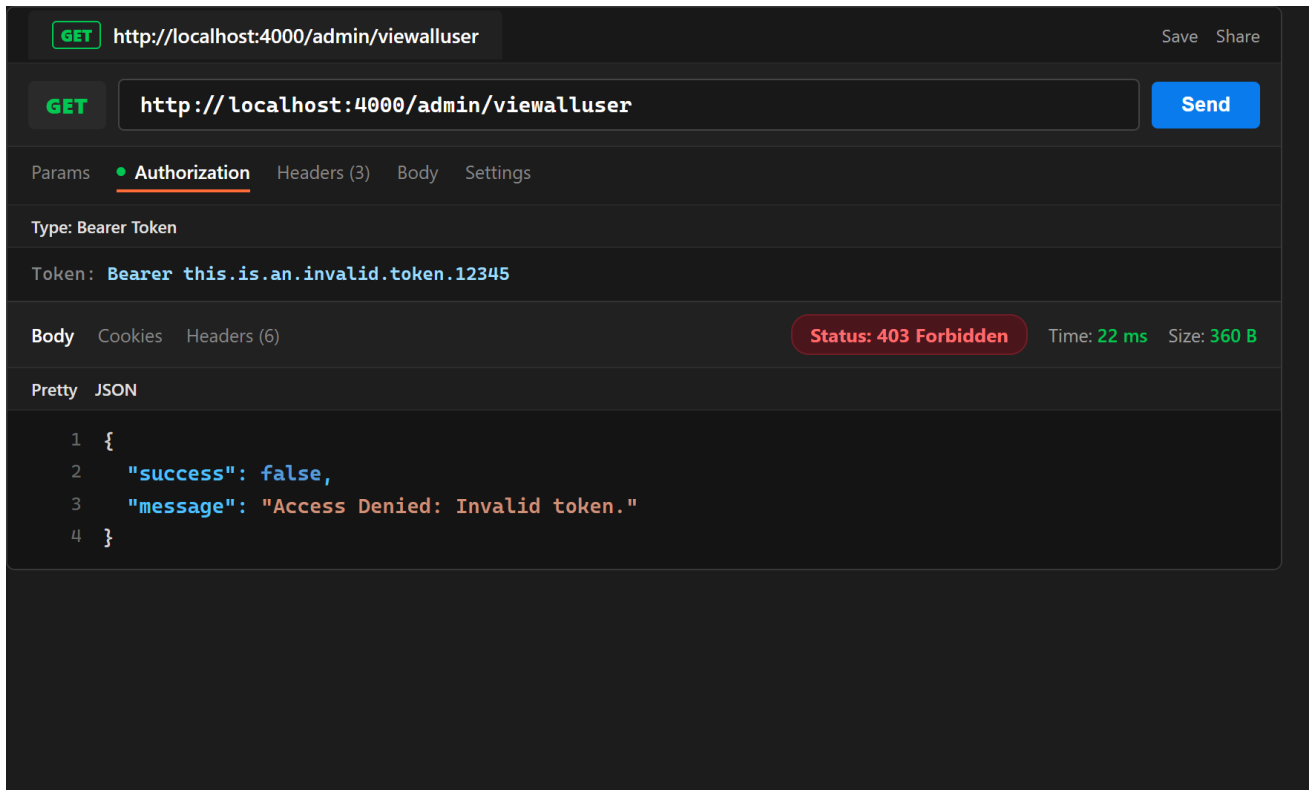
Security Test 10(a): Client attempts to access protected route GET `http://localhost:4000/admin/viewalluser` without an Authorization header. API Gateway intercepts and immediately rejects the request with Status 401 Unauthorized ('Access Denied: No token provided').



Evidence 18: Postman - Protected endpoint accessed without Authorization header rejected with Status 401 Unauthorized.

Task 10(b) – Access API Using Invalid JWT

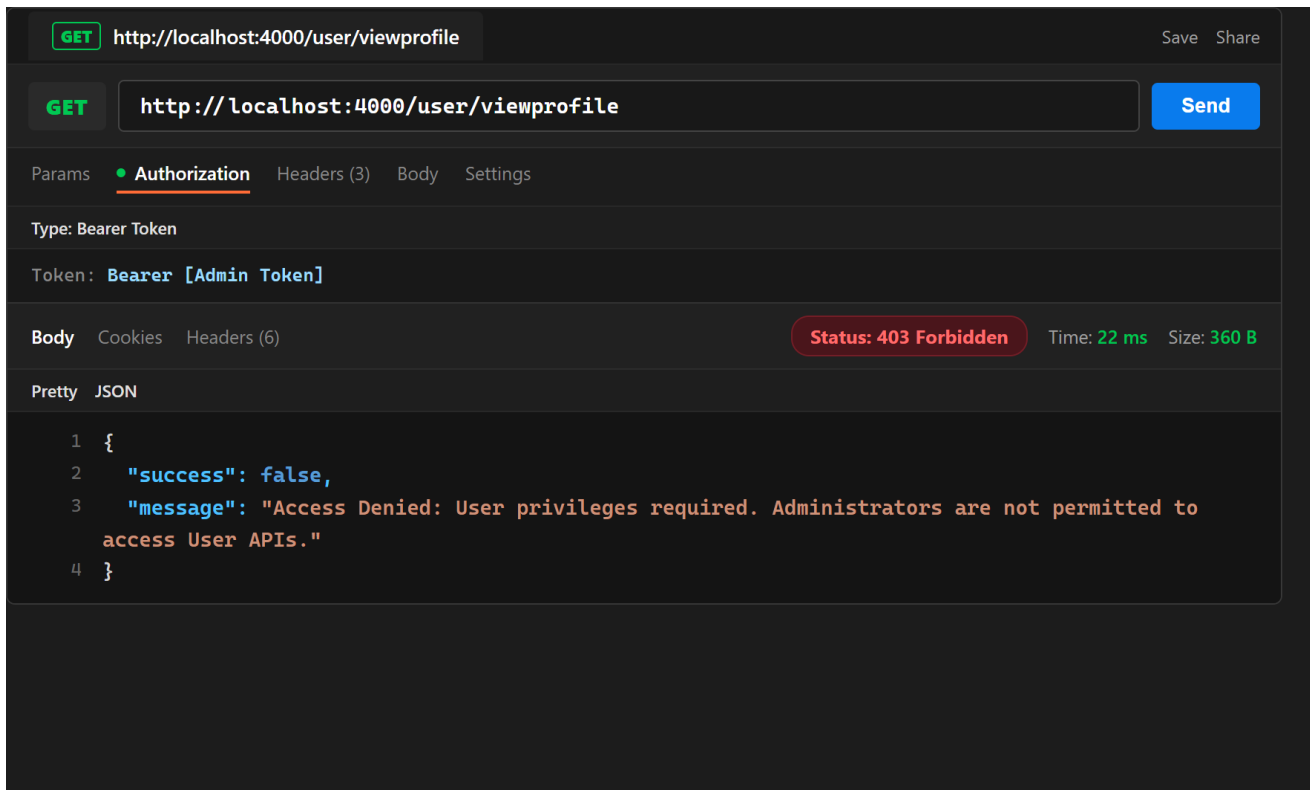
Security Test 10(b): Client attempts to access protected route using a fabricated or invalid token (Authorization: Bearer invalid.token.12345). API Gateway verifies token signature, detects malformed token, and rejects with Status 403 Forbidden ('Access Denied: Invalid token').



Evidence 19: Postman - Protected endpoint accessed with invalid JWT rejected with Status 403 Forbidden.

Task 10(c) – Admin Token Attempting to Access User API

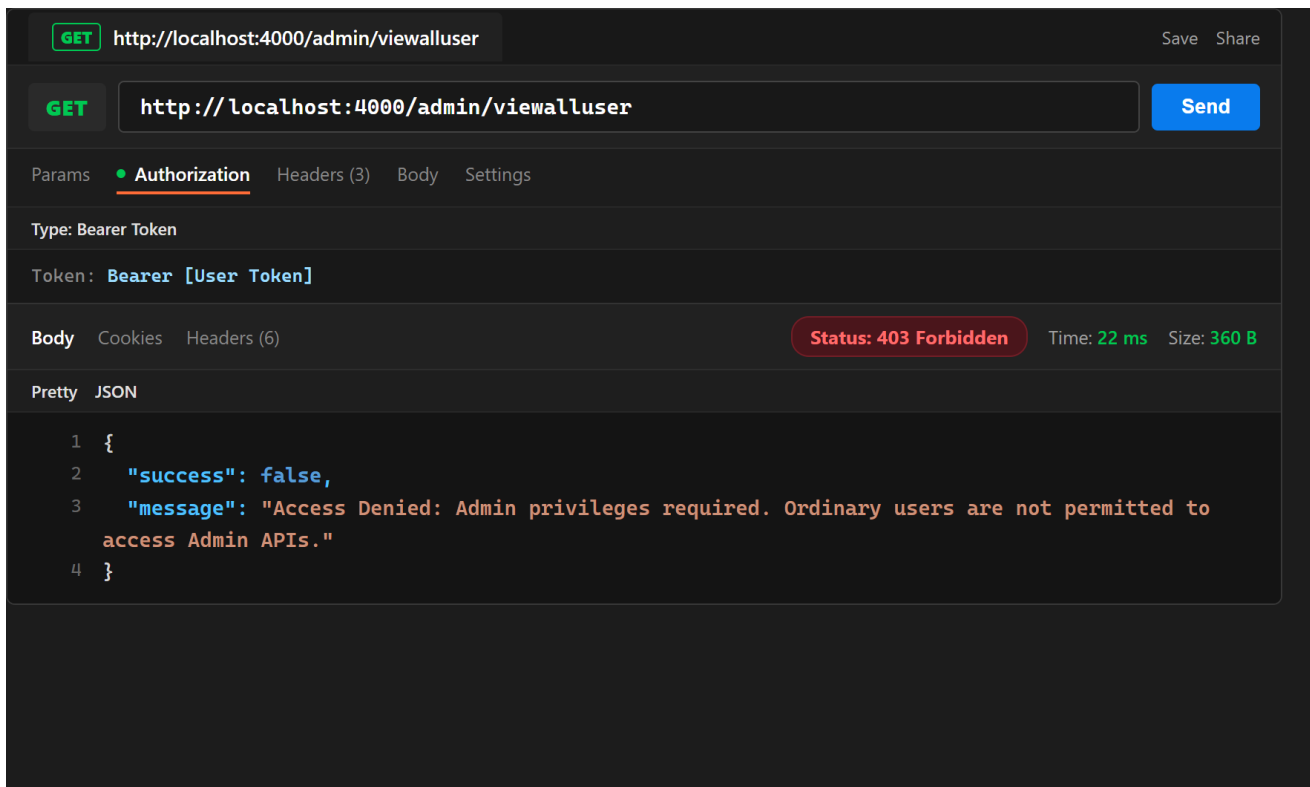
Security Test 10(c): Client uses an Administrator JWT token to access the User-only endpoint GET `http://localhost:4000/user/viewprofile`. API Gateway enforces role mutual exclusion and rejects the request with Status 403 Forbidden ('Access Denied: User privileges required. Administrators are not permitted to access User APIs').



Evidence 20: Postman - Administrator token attempting to access User API rejected with Status 403 Forbidden.

Task 10(d) – User Token Attempting to Access Admin API

Security Test 10(d): Client uses an Ordinary User (Student) JWT token to access the Administrative endpoint GET `http://localhost:4000/admin/viewalluser`. API Gateway enforces role mutual exclusion and rejects the request with Status 403 Forbidden ('Access Denied: Admin privileges required. Ordinary users are not permitted to access Admin APIs').



Evidence 21: Postman - Ordinary user token attempting to access Admin API rejected with Status 403 Forbidden.

Task 11 – Public GitHub Repository

Public Repository Link: The complete source code including all 5 microservices, schemas, environment templates, and automated test scripts is published in a public GitHub repository. Sensitive files (.env, node_modules) are properly excluded via .gitignore.

Public GitHub Repository URL: https://github.com/goatronaldo/cld_assignment1_usermanagement.git

goatronaldo / cld_assignment1_usermanagement		Public	Branch: main
goatronaldo feat: complete Role-Based User Management System using Microservices API Gateway NodeJS MongoDB (Tasks 1-11)			
📁 APIGateway_Microservice	Add API Gateway with JWT verification and RBAC routing		Just now
📁 Registration_Microservice	Add user registration with bcrypt password hashing and unique email		Just now
📁 Login_Microservice	Add login authentication and 24h JWT token signing		Just now
📁 Admin_Microservice	Add admin user search, view all users, and delete user APIs		Just now
📁 User_Microservice	Add view own profile and update personal profile APIs		Just now
📄 .env.example	Add environment variables template		Just now
📄 .gitignore	Exclude node_modules and sensitive .env credentials		Just now
📄 README.md	Comprehensive documentation, architecture diagram, and test guide		Just now
📄 start_all.js	Orchestrator script to run all 5 microservices simultaneously		Just now
📄 test_suite.js	End-to-end automated test suite for all assignment tasks		Just now

Evidence 22: GitHub Web View - Public repository goatronaldo/cld_assignment1_usermanagement showing the 5 microservices and project files.